MODULE *Overwatch*

Section: Imports

Shared operators used by this module are imported below.

EXTENDS *Naturals*, *FiniteSets*, *Sequences*, *FailurePolicy*

Section: State and result domains

Workspace versions, completed outcomes, and message records form the result layer. Finite bounds are supplied by the *TLC* configuration, not by the runtime.

CONSTANTS *Names*, *MaxEdits*, *Timeout*

ASSUME "timeout" $\notin$ *Names*

$Versions \triangleq 0 \,..\, MaxEdits$

$Results \triangleq \text{SUBSET } Names$

$TimedOut \triangleq$ "timeout"

$Outcomes \triangleq \text{SUBSET } (Names \cup \{TimedOut\})$

$NoMessage \triangleq [held \mapsto \text{FALSE},\ waking \mapsto \text{FALSE},\ failures \mapsto \{\}]$

$NotTold \triangleq [kind \mapsto \text{"none"},\ fresh \mapsto \{\}]$

$NoBaseline \triangleq [known \mapsto \text{FALSE},\ at \mapsto 0,\ failures \mapsto \{\}]$

Section: State shape

The checked version, unread results, inherited baseline, and delivery state are separate. touched records whether the session has edited since the commit the baseline belongs to; own is the versions those edits made, which only the properties read. *TypeOK* constrains their domains without advancing an execution.

VARIABLES
truth, *head*, *based*, *busy*, *from*, *quiet*, *installed*,
produced, *consumed*, *pending*, *report*, *baseline*, *wasFailing*,
outbox, *told*, *flush*, *turn*, *toldThisTurn*, *promptQueued*, *knows*, *elapsed*, *edited*,
touched, *own*

$vars \triangleq \langle truth, head, based, busy, from, quiet, installed,$
$produced, consumed, pending, report, baseline, wasFailing,$
$outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed, edited,$
$touched, own\rangle$

$TypeOK \triangleq$
$\land\ truth \in [Versions \to Results]$
$\land\ head \in Versions \land based \in 0 \,..\, head$
$\land\ busy \in \text{BOOLEAN} \land from \in Versions \land quiet \in \text{BOOLEAN} \land installed \in \text{BOOLEAN}$
$\land\ produced \in 0 \,..\, (MaxEdits + 1) \land consumed \in 0 \,..\, produced$
$\land\ pending \in Seq([id : 1 \,..\, (MaxEdits + 1),\ at : Versions,\ failures : Outcomes])$
$\land\ Len(pending) \leq MaxEdits + 1$
$\land\ report \in [known : \text{BOOLEAN},\ id : 0 \,..\, produced,\ at : Versions,\ failures : Outcomes]$
$\land\ baseline \in [known : \text{BOOLEAN},\ at : Versions,\ failures : Outcomes]$
$\land\ wasFailing \in \text{BOOLEAN}$
$\land\ outbox \in [held : \text{BOOLEAN},\ waking : \text{BOOLEAN},\ failures : Outcomes]$
$\land\ told \in [kind : \{\text{"none"}, \text{"failed"}, \text{"recovered"}\},\ fresh : Outcomes]$
$\land\ flush \in \text{BOOLEAN}$
$\land\ turn \in \{\text{"idle"}, \text{"running"}\}$
$\land\ toldThisTurn \in \text{BOOLEAN} \land promptQueued \in \text{BOOLEAN}$
$\land\ knows \in Outcomes \land elapsed \in 0 \,..\, Timeout$
$\land\ edited \in \text{BOOLEAN}$
$\land\ touched \in \text{BOOLEAN} \land own \subseteq Versions$

Section: Failure gating

A failure report becomes a gate only when it is a regression against the session baseline, or when the check times out.

$$Failing \triangleq report.known \land report.failures \neq \{\}$$
$$Regressed(f) \triangleq TimedOut \in f \lor (baseline.known \land NewFailure(f,\ baseline.failures))$$
$$Guard \triangleq Failing \land Regressed(report.failures)$$

Section: Initial state

The first check starts at the inherited workspace version. No result or baseline is known and no notification has been sent.

$$\begin{aligned}
Init \triangleq\ & \land truth \in [Versions \to Results] \\
& \land head = 0 \land based = 0 \\
& \land busy = \text{TRUE} \land from = 0 \land quiet = \text{FALSE} \land installed = \text{FALSE} \\
& \land produced = 0 \land consumed = 0 \land pending = \langle\rangle \\
& \land report = [known \mapsto \text{FALSE},\ id \mapsto 0,\ at \mapsto 0,\ failures \mapsto \{\}] \\
& \land baseline = NoBaseline \\
& \land wasFailing = \text{FALSE} \\
& \land outbox = NoMessage \land told = NotTold \land flush = \text{FALSE} \\
& \land turn = \text{“idle”} \land toldThisTurn = \text{FALSE} \land promptQueued = \text{FALSE} \\
& \land knows = \{\} \land elapsed = 0 \land edited = \text{FALSE} \\
& \land touched = \text{FALSE} \land own = \{\}
\end{aligned}$$

Section: Result delivery and baseline isolation

Queue deduplicates notification signatures. Deliver consumes a completed snapshot, establishes the inherited baseline, and queues a regression or recovery. The first result is the baseline unless the session has edited since the commit, as another process's changes are inherited like the commit itself; after a session edit the baseline is empty, so no failure the edit may hold is absorbed. A recovery can cancel a failure message that has not yet been delivered.

$$\begin{aligned}
& Queue(signature,\ message) \triangleq \\
& \quad \text{IF } told = signature \land \neg outbox.held \\
& \quad \text{THEN UNCHANGED } \langle outbox,\ told,\ flush\rangle \\
& \quad \text{ELSE } \land told' = signature \land outbox' = message \land flush' = \text{TRUE}
\end{aligned}$$

$$\begin{aligned}
& Deliver(v,\ id,\ f) \triangleq \\
& \quad \text{LET } b \triangleq \text{IF } baseline.known \text{ THEN } baseline.failures \text{ ELSE IF } \neg touched \text{ THEN } f \text{ ELSE } \{\} \\
& \qquad isRegression \triangleq TimedOut \in f \lor \neg(f \subseteq b) \\
& \quad \text{IN } \land report' = [known \mapsto \text{TRUE},\ id \mapsto id,\ at \mapsto v,\ failures \mapsto f] \\
& \qquad \land baseline' = \text{IF } TimedOut \in f \lor baseline.known \text{ THEN } baseline \\
& \qquad\qquad\qquad \text{ELSE } [known \mapsto \text{TRUE},\ at \mapsto v,\ failures \mapsto b] \\
& \qquad \land wasFailing' = isRegression \\
& \qquad \land \text{IF } isRegression \\
& \qquad\quad \text{THEN } Queue([kind \mapsto \text{“failed”},\ fresh \mapsto f \setminus b], \\
& \qquad\qquad\qquad [held \mapsto \text{TRUE},\ waking \mapsto edited \lor TimedOut \in f,\ failures \mapsto f]) \\
& \qquad\quad \text{ELSE IF } \neg wasFailing \\
& \qquad\qquad \text{THEN UNCHANGED } \langle outbox,\ told,\ flush\rangle \\
& \qquad\quad \text{ELSE IF } outbox.held \\
& \qquad\qquad \text{THEN } \land outbox' = NoMessage \land told' = NotTold \land \text{UNCHANGED } flush \\
& \qquad\quad \text{ELSE } Queue([kind \mapsto \text{“recovered”},\ fresh \mapsto \{\}],\ [held \mapsto \text{TRUE},\ waking \mapsto \text{FALSE},\ failures \mapsto f])
\end{aligned}$$

Section: Workspace changes and scheduling

An agent edit advances head without moving based, and marks the next read as following the agent's own change and the session as having edited. Another process can change the workspace at any time without committing; *OutsideEdit* advances head the same way but marks nothing. An external head change moves both, invalidates the baseline and unread results, and starts the session's edits afresh. *QuietElapses* starts the next check; *Tick* advances its timeout clock.

$$\begin{aligned}
Edit \triangleq\ & \land turn = \text{“running”} \land head < MaxEdits \\
& \land head' = head + 1 \land quiet' = \text{TRUE} \land edited' = \text{TRUE} \\
& \land touched' = \text{TRUE} \land own' = own \cup \{head + 1\} \\
& \land \text{UNCHANGED } \langle truth,\ based,\ busy,\ from,\ installed,\ produced,\ consumed,\ pending, \\
& \qquad\qquad report,\ baseline,\ wasFailing,\ outbox,\ told,\ flush,\ turn, \\
& \qquad\qquad toldThisTurn,\ promptQueued,\ knows,\ elapsed\rangle
\end{aligned}$$

$$
\begin{aligned}
&\mathit{OutsideEdit} \triangleq \\
&\quad \land\ \mathit{head} < \mathit{MaxEdits} \\
&\quad \land\ \mathit{head}' = \mathit{head} + 1 \land \mathit{quiet}' = \text{TRUE} \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{based}, \mathit{busy}, \mathit{from}, \mathit{installed}, \mathit{produced}, \mathit{consumed}, \mathit{pending}, \\
&\qquad\qquad \mathit{report}, \mathit{baseline}, \mathit{wasFailing}, \mathit{outbox}, \mathit{told}, \mathit{flush}, \mathit{turn}, \\
&\qquad\qquad \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{elapsed}, \mathit{edited}, \mathit{touched}, \mathit{own} \rangle
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{HeadMoves} \triangleq \\
&\quad \land\ \mathit{head} < \mathit{MaxEdits} \\
&\quad \land\ \mathit{head}' = \mathit{head} + 1 \land \mathit{based}' = \mathit{head} + 1 \\
&\quad \land\ \mathit{baseline}' = \mathit{NoBaseline} \land \mathit{touched}' = \text{FALSE} \land \mathit{own}' = \{\} \\
&\quad \land\ \mathit{quiet}' = \text{TRUE} \land \mathit{pending}' = \langle\rangle \\
&\quad \land\ \mathit{from}' = \text{IF } \mathit{busy} \text{ THEN } \mathit{head} + 1 \text{ ELSE } \mathit{from} \\
&\quad \land\ \mathit{elapsed}' = \text{IF } \mathit{busy} \text{ THEN } 0 \text{ ELSE } \mathit{elapsed} \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{busy}, \mathit{installed}, \mathit{produced}, \mathit{consumed}, \mathit{report}, \mathit{wasFailing}, \mathit{outbox}, \\
&\qquad\qquad \mathit{told}, \mathit{flush}, \mathit{turn}, \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{edited} \rangle
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{QuietElapses} \triangleq \\
&\quad \land\ \mathit{quiet} \land \mathit{installed} \\
&\quad \land\ \mathit{quiet}' = \text{FALSE} \land \mathit{busy}' = \text{TRUE} \land \mathit{from}' = \mathit{head} \land \mathit{elapsed}' = 0 \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{head}, \mathit{based}, \mathit{installed}, \mathit{produced}, \mathit{consumed}, \mathit{pending}, \\
&\qquad\qquad \mathit{report}, \mathit{baseline}, \mathit{wasFailing}, \mathit{outbox}, \mathit{told}, \mathit{flush}, \mathit{turn}, \\
&\qquad\qquad \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{edited}, \mathit{touched}, \mathit{own} \rangle
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{Tick} \triangleq \\
&\quad \land\ \mathit{busy} \land \mathit{elapsed} < \mathit{Timeout} \land \mathit{elapsed}' = \mathit{elapsed} + 1 \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{head}, \mathit{based}, \mathit{busy}, \mathit{from}, \mathit{quiet}, \mathit{installed}, \mathit{produced}, \\
&\qquad\qquad \mathit{consumed}, \mathit{pending}, \mathit{report}, \mathit{baseline}, \mathit{wasFailing}, \mathit{outbox}, \\
&\qquad\qquad \mathit{told}, \mathit{flush}, \mathit{turn}, \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{edited}, \\
&\qquad\qquad \mathit{touched}, \mathit{own} \rangle
\end{aligned}
$$

Completion appends a result; it does not deliver it. Partial output is a stuttering step at this abstraction: only a complete build enters pending.

Section: Buffered completion and consumption

Run completion appends a snapshot to pending without publishing it. *ReadResults* consumes the newest completed snapshot and clears the buffer. A later incomplete build does not replace an earlier complete result.

$$
\begin{aligned}
&\mathit{Buffer}(v, f) \triangleq \\
&\quad \land\ \mathit{produced}' = \mathit{produced} + 1 \\
&\quad \land\ \mathit{pending}' = \mathit{Append}(\mathit{pending}, [\mathit{id} \mapsto \mathit{produced} + 1, \mathit{at} \mapsto v, \mathit{failures} \mapsto f]) \\
&\quad \land\ \mathit{busy}' = \text{FALSE} \land \mathit{installed}' = \text{TRUE} \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{head}, \mathit{based}, \mathit{from}, \mathit{quiet}, \mathit{consumed}, \mathit{report}, \mathit{baseline}, \\
&\qquad\qquad \mathit{wasFailing}, \mathit{outbox}, \mathit{told}, \mathit{flush}, \mathit{turn}, \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{elapsed}, \mathit{edited}, \\
&\qquad\qquad \mathit{touched}, \mathit{own} \rangle
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{RunEnds} \triangleq \mathit{busy} \land \exists\, v \in \mathit{from} \,..\, \mathit{head} : \mathit{Buffer}(v, \mathit{truth}[v]) \\
&\mathit{TimesOut} \triangleq \mathit{busy} \land \mathit{elapsed} = \mathit{Timeout} \land \mathit{Buffer}(\mathit{from}, \{\mathit{TimedOut}\})
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{ReadResults} \triangleq \\
&\quad \land\ \mathit{pending} \neq \langle\rangle \\
&\quad \land\ \text{LET } \mathit{latest} \triangleq \mathit{pending}[\mathit{Len}(\mathit{pending})] \\
&\qquad \text{IN } \mathit{Deliver}(\mathit{latest.at}, \mathit{latest.id}, \mathit{latest.failures}) \\
&\quad \land\ \mathit{consumed}' = \mathit{produced} \land \mathit{pending}' = \langle\rangle \land \mathit{edited}' = \text{FALSE} \\
&\quad \land\ \text{UNCHANGED } \langle \mathit{truth}, \mathit{head}, \mathit{based}, \mathit{busy}, \mathit{from}, \mathit{quiet}, \mathit{installed}, \mathit{produced}, \\
&\qquad\qquad \mathit{turn}, \mathit{toldThisTurn}, \mathit{promptQueued}, \mathit{knows}, \mathit{elapsed}, \mathit{touched}, \mathit{own} \rangle
\end{aligned}
$$

Section: Notification delivery and turn completion

A running turn receives at most one notification before another turn starts. An idle agent can receive one queued wakeup. Flush publishes eligible messages; *TurnCompletes* keeps a guarded turn running and permits a clean turn to settle.

$$Held \triangleq \text{IF } turn = \text{“running”} \text{ THEN } toldThisTurn \text{ ELSE } promptQueued \vee \neg outbox.waking$$

$$\begin{aligned}
&Flush \triangleq \\
&\quad \wedge flush \wedge flush' = \text{FALSE} \\
&\quad \wedge \text{IF } outbox.held \wedge \neg Held \\
&\qquad \text{THEN } \wedge outbox' = NoMessage \wedge knows' = outbox.failures \\
&\qquad\quad \wedge \text{IF } turn = \text{“running”} \\
&\qquad\qquad \text{THEN } toldThisTurn' = \text{TRUE} \wedge \text{UNCHANGED } promptQueued \\
&\qquad\qquad \text{ELSE } promptQueued' = \text{TRUE} \wedge \text{UNCHANGED } toldThisTurn \\
&\qquad \text{ELSE } \text{UNCHANGED } \langle outbox, knows, toldThisTurn, promptQueued \rangle \\
&\quad \wedge \text{UNCHANGED } \langle truth, head, based, busy, from, quiet, installed, produced, consumed, \\
&\qquad\qquad pending, report, baseline, wasFailing, told, turn, elapsed, edited, \\
&\qquad\qquad touched, own \rangle
\end{aligned}$$

$$\begin{aligned}
&TurnStarts \triangleq \\
&\quad \wedge turn = \text{“idle”} \wedge turn' = \text{“running”} \\
&\quad \wedge toldThisTurn' = promptQueued \wedge promptQueued' = \text{FALSE} \wedge flush' = \text{TRUE} \\
&\quad \wedge \text{UNCHANGED } \langle truth, head, based, busy, from, quiet, installed, produced, consumed, \\
&\qquad\qquad pending, report, baseline, wasFailing, outbox, told, knows, elapsed, edited, \\
&\qquad\qquad touched, own \rangle
\end{aligned}$$

$$WokenTurnStarts \triangleq promptQueued \wedge TurnStarts$$

$$\begin{aligned}
&TurnCompletes \triangleq \\
&\quad \wedge turn = \text{“running”} \\
&\quad \wedge \text{IF } Guard \\
&\qquad \text{THEN } \wedge toldThisTurn' = \text{TRUE} \wedge knows' = report.failures \wedge \text{UNCHANGED } \langle turn, flush \rangle \\
&\qquad \text{ELSE } \wedge turn' = \text{“idle”} \wedge flush' = \text{TRUE} \wedge \text{UNCHANGED } \langle toldThisTurn, knows \rangle \\
&\quad \wedge \text{UNCHANGED } \langle truth, head, based, busy, from, quiet, installed, produced, consumed, \\
&\qquad\qquad pending, report, baseline, wasFailing, outbox, told, promptQueued, elapsed, edited, \\
&\qquad\qquad touched, own \rangle
\end{aligned}$$

Section: Allowed executions and fairness

Next chooses an enabled transition. Spec permits stuttering and requires weak fairness for the listed progress actions; this does not impose deadlines.

$$\begin{aligned}
&Next \triangleq Edit \vee OutsideEdit \vee HeadMoves \vee QuietElapses \vee Tick \vee RunEnds \vee TimesOut \\
&\qquad \vee ReadResults \vee Flush \vee TurnStarts \vee TurnCompletes
\end{aligned}$$

$$\begin{aligned}
&Spec \triangleq \\
&\quad \wedge Init \wedge \Box[Next]_{vars} \\
&\quad \wedge \mathrm{WF}_{vars}(QuietElapses) \wedge \mathrm{WF}_{vars}(Tick) \wedge \mathrm{WF}_{vars}(TimesOut) \\
&\quad \wedge \mathrm{WF}_{vars}(ReadResults) \wedge \mathrm{WF}_{vars}(Flush) \\
&\quad \wedge \mathrm{WF}_{vars}(WokenTurnStarts) \wedge \mathrm{WF}_{vars}(TurnCompletes)
\end{aligned}$$

Section: Safety and progress properties

Results retain their ordering and checked version. The baseline cannot absorb the session's edits: a non-empty baseline is exactly the failures of a version at or after the commit that no session edit made or preceded. A failure wakes the agent only when its read follows the agent's own edit, or when the check timed out, so another process's changes never wake it. Under *Spec* fairness, checks end, and each regression, a failure outside the baseline taken, reaches the agent or waits, without waking it, for its next turn. These properties are model-checked within the committed configuration bounds.

$$\begin{aligned}
&BufferedResultsAreOrdered \triangleq \\
&\quad \wedge \forall\, i \in 1 \,..\, Len(pending) : pending[i].id > consumed \wedge pending[i].id \leq produced \\
&\quad \wedge \forall\, i \in 1 \,..\, (Len(pending) - 1) : pending[i].id < pending[i+1].id
\end{aligned}$$

$$ReadsSelectLatestComplete \triangleq report.known \Rightarrow report.id = consumed$$

$$ResultsMatchBuilds \triangleq report.known \Rightarrow report.failures = truth[report.at] \vee report.failures = \{TimedOut\}$$

$$\begin{aligned}
&BaselineNeverAbsorbsEdits \triangleq \\
&\quad baseline.known \Rightarrow baseline.failures = \{\} \vee \\
&\qquad \wedge baseline.failures = truth[baseline.at] \wedge baseline.at \geq based
\end{aligned}$$

```
265             ∧ ∀ w ∈ own : w > baseline.at
266   RunsEnd ≜ busy ⇝ ¬busy
267   EveryEditIsChecked ≜ ◇□(report.known ∧ report.at = head)
268   Regressions ≜ truth[head] \ IF baseline.known THEN baseline.failures ELSE truth[based]
269   WaitsForTurn ≜ turn = "idle" ∧ outbox.held ∧ ¬outbox.waking ∧ Regressions ⊆ outbox.failures
270   RegressionsReachAgent ≜ ◇□(Regressions ⊆ knows ∨ TimedOut ∈ knows ∨ WaitsForTurn)
271   OutsideChangesNeverWake ≜
272       □[told′ ≠ told ∧ told′.kind = "failed" ∧ outbox′.waking
273           ⇒ edited ∨ TimedOut ∈ outbox′.failures]_vars
274 ═══════════════════════════════════════════════════════════════
```